

DP Algorithms & Model-Free RL

Solving Known MDPs: Dynamic Programming

1) Policy Evaluation

Policy evaluation is the process of calculating the state-value function ($V^\pi(s)$). It tells us the goodness or badness of a certain policy (π). For this, we use the Bellman Expectation Equation for the state-value function:

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r | s,a) [r + \gamma V_k(s')]$$

Pseudocode:

- **Input:**
 - The policy π which we want to evaluate.
 - An estimation threshold $\theta > 0$ to determine the accuracy of our estimation.
 - Transition probabilities / Transition Matrix.
 - Discount factor $\gamma \in [0, 1]$.
- **Process:**
 - Set the value of every state arbitrarily.
 - Set the value of the terminal state to 0.
 - Set Δ to 0.
 - **Loop** until $\Delta < \theta$:
 - For each state in the state space:
 - Store the current value of the state in a variable.
 - Update the value of the state using the Bellman Expectation Equation.
 - Check how much the state's value changed.
 - Set $\Delta \leftarrow \max(\text{current } \Delta, \text{absolute difference between the old and new state value})$.
- **Output:** The final value of each state, which is now a close approximation of the value function $V^\pi(s)$ for the given policy.

2) Policy Iteration

Policy Iteration is an algorithm used to discover the optimal policy.

Pseudocode:

- **Input:**
 - An estimation threshold $\theta > 0$ to determine the accuracy of our estimation.

- Transition probabilities / Transition Matrix.
- Discount factor $\gamma \in [0, 1]$.
- **Process:**
 - Set the value of every state arbitrarily.
 - Set Δ to 0.
 - Start with an arbitrary policy and perform **Policy Evaluation**.
 - Then perform **Policy Improvement**, for each state:
 - Store the current action and calculate the expected return for all possible actions.
 - Select the action with the highest expected return.
 - Update the policy to choose this action.
 - If the selected action differs from the previous action, mark the policy as unstable.
 - Check whether the policy changed: If no action changed in any state, stop; otherwise, repeat Policy Evaluation with the improved policy.
- **Output:** Optimal policy π^* and the optimal value function $V^*(s)$ corresponding to that policy.

3) Value Iteration

Value Iteration is an algorithm which helps in finding the optimal strategy faster. For this, we use the Bellman Optimality Equation of the state-value function:

$$V_{k+1}(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma V_k(s')]$$

Pseudocode:

- **Input:**
 - An estimation threshold $\theta > 0$ to determine the accuracy of our estimation.
 - Transition probabilities / Transition Matrix.
 - Discount factor $\gamma \in [0, 1]$.
- **Process:**
 - Set the value of every state arbitrarily.
 - Set the value of the terminal state to 0.
 - **Loop** until the maximum change in state values (Δ) becomes smaller than θ :
 - Set Δ to 0.
 - Repeatedly update the value of each state using the Bellman Optimality Equation.
 - Extract the optimal policy by choosing, in each state, the action that yields the highest expected return according to the converged value function.
- **Output:** Optimal policy π^* and the optimal value function $V^*(s)$ corresponding to that policy.

Introduction to Model-Free RL: MC vs TD

We will take an example of a small toy MDP for this comparison to highlight how learning from experience differs from Dynamic Programming.

Toy MDP Setup

Imagine a 2×2 grid:

- Top Left (TL) $\rightarrow$ Top Right (TR)
- Bottom Left (BL) $\rightarrow$ Bottom Right (BR)

BR is our Goal.

- **Actions:** Up, Down, Left, Right.
- **Rewards:** Every step taken costs -1 & stepping into goal (BR) gives $+10$.
- **Discount factor (γ):** 1 (i.e., no discounting).
- **Learning Rate (α):** 0.5.

Initially, $V(TL) = 0$, $V(TR) = 0$, $V(BL) = 0$, $V(BR) = 0$.

Let's simulate one specific episode starting at BL:

$$BL \xrightarrow{\text{UP}} TL \ (R = -1) \xrightarrow{\text{RIGHT}} TR \ (R = -1) \xrightarrow{\text{DOWN}} BR \ (R = +10)$$

Now compare between Monte Carlo and Temporal Difference algorithms:

1) Monte Carlo (MC) Execution

MC must wait until it hits a terminal state as it calculates cumulative return (G_t) from each state.

Work Backwards:

- From TR $\rightarrow$ only one step and total reward is 10. $\therefore G_{TR} = 10$.
- From TL $\rightarrow G_{TL} = (-1) + 10 = 9$.
- From BL $\rightarrow G_{BL} = (-1) + (-1) + 10 = 8$.

Now update the values:

- **Update TR** $\rightarrow V(TR) \leftarrow V(TR) + \alpha[G_{TR} - V(TR)]$
 $V(TR) \leftarrow 0 + 0.5[10 - 0] = 5$
- **Update TL** $\rightarrow V(TL) \leftarrow V(TL) + \alpha[G_{TL} - V(TL)]$
 $V(TL) \leftarrow 0 + 0.5[9 - 0] = 4.5$

- **Update BL** $\rightarrow V(BL) \leftarrow V(BL) + \alpha[G_{BL} - V(BL)]$
 $V(BL) \leftarrow 0 + 0.5[8 - 0] = 4$

We got: $V(TR) = 5$, $V(TL) = 4.5$, $V(BL) = 4$.

2) Temporal Difference (TD) Execution

TD updates the value immediately after every move, bootstrapping off the next state's current estimate.

- **First step (BL $\rightarrow$ TL)** $\rightarrow R + \gamma V(TL) = -1 + 1(0) = -1$
 $V(BL) \leftarrow V(BL) + 0.5[-1 - 0] = -0.5$
- **Second step (TL $\rightarrow$ TR)** $\rightarrow R + \gamma V(TR) = -1 + 1(0) = -1$
 $V(TL) \leftarrow V(TL) + 0.5[-1 - 0] = -0.5$
- **Third step (TR $\rightarrow$ BR)** $\rightarrow R + \gamma V(BR) = 10 + 1(0) = 10$
 $V(TR) \leftarrow V(TR) + 0.5[10 - 0] = 5$

We got: $V(TR) = 5$, $V(TL) = -0.5$, $V(BL) = -0.5$.

Value Comparison

	MC	TD
$V(TR)$	5	5
$V(TL)$	4.5	-0.5
$V(BL)$	4	-0.5